

Socket-API

Standardisierte Betriebssystemschnittstelle zur Kommunikation über (TCP,UDP)/IP Netze

- Unterstützt sowohl verbindungsorientierte und paketorientierte Kommunikation sowie Multit-/Broadcast-Kommunikation
- Unterste Schnittstelle zum Betriebssystem
- Synchronisation vollständig beim Anwender
- Fehlerbehandlung liegt ebenfalls beim Anwender
 - Keine Unterstützung für weitere Fehlertoleranz
 - Fehlererkennung außerhalb von TCP geleisteter Flusskontrolle sowie Timeouts
- Nicht-binär-transparente Datenübertragung
- Unterstützt daher nicht die Übertragung komplexer Strukturen bzw. Objekten

Anwendungsprotokolle

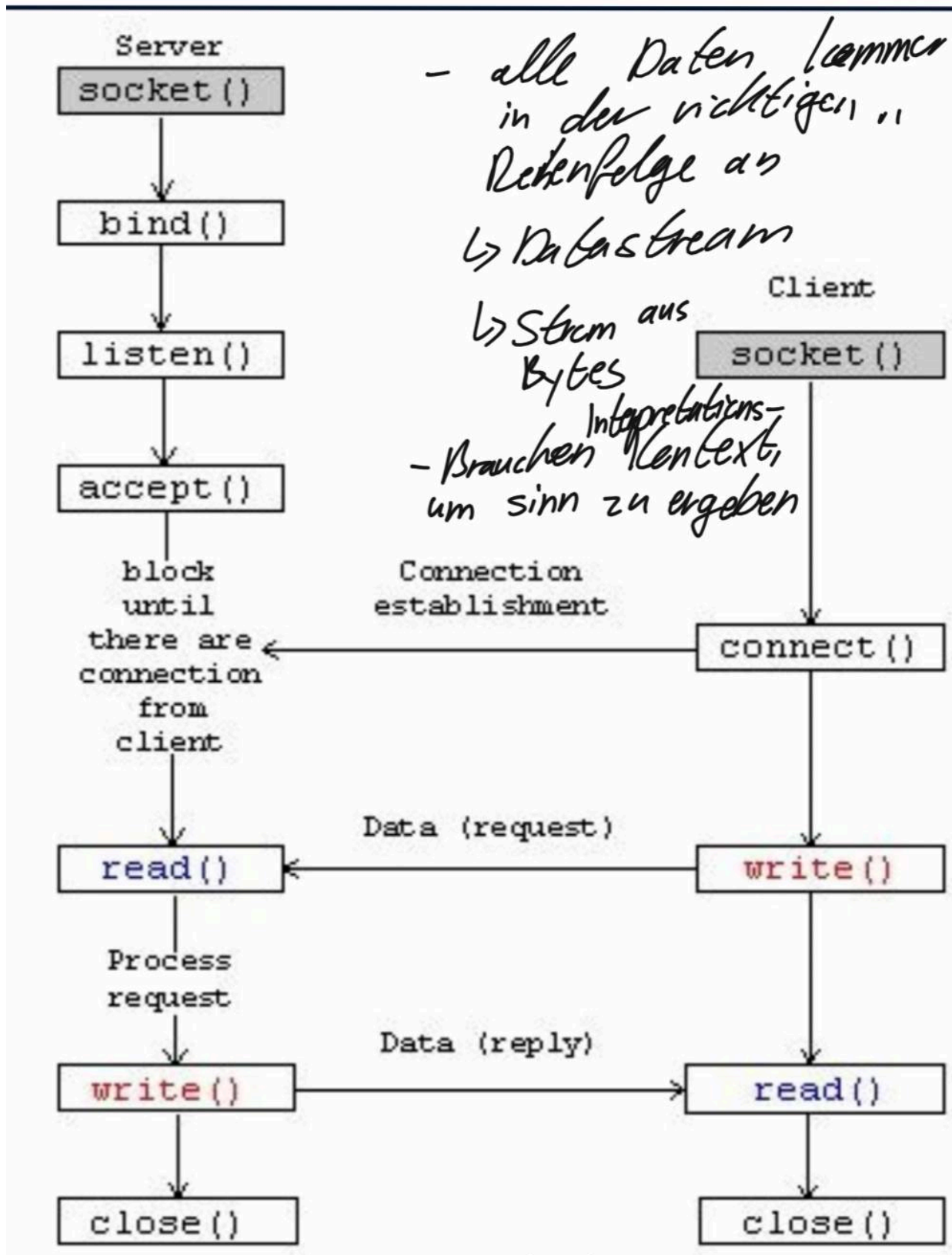
Übernahme von exakter Codierung der Nachrichten sowie der Spezifizierung des Ablaufs der Kommunikation und Definierung für Fehlerbehandlung

Beispiele: SMTP, HTTP, HTTPS, HTTP/2

Verbindungsorientierte Kommunikation

- TCP
- Funktioniert solange gut, bis der Client langsam ist
- Telefonprinzip
- Server wartet in `accept()`
- Client stellt Verbindung her
- Server bekommt mit jeder neuen Verbindung eine neue Socket für Kommunikation mit diesem einen Client
- Datenstrombasierte Kommunikation über `send (write)` `recv (read)`

- Kommunikationspartner werden über Fehler informiert

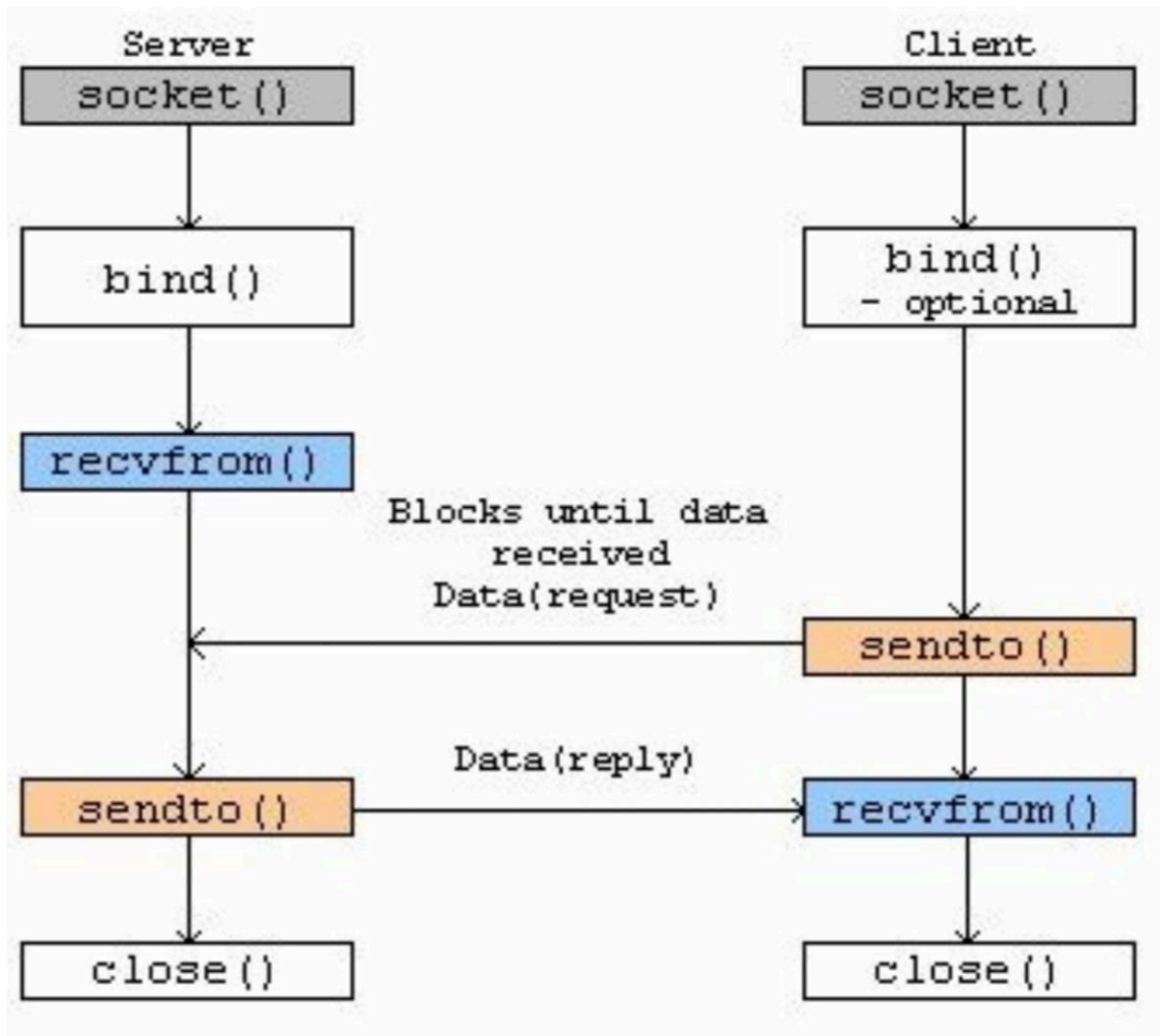


Paket-orientierte Kommunikation

UDP

- Client und Server öffnen UDP-Socket
- In `sendto()` bzw. `recvfrom()` Aufrufen werden
 - Abgeschlossene Datenpakete gesendet bzw. empfangen

- Bei jedem Aufruf werden Ziel- und Empfängeradresse kommuniziert
- System meldet
 - Weder Kommunikationsfehler
 - noch das Ankommen von Paketen und deren Reihenfolge



Übertragung komplexer binärer Datenstrukturen

Problem mit Sockets:

- Bei komplexen Datenstrukturen kann die Anordnung einzelner Attribute auf vielfache von 2 Bytes ausgerichtet sein/ Padding
- Oder mit Pointern im Speicher verteilt
- Endianness/ Verteilung von Daten auf mehrere Adressen
- Socket kann das nicht

Externe Datenrepräsentation

gemeinsames Datenformat zur transparenten Übertragung von beliebigen (binären) Daten in ansonsten heterogenen Umgebungen

En- und Dekodierung

- **Marshalling:** Prozess der Transformation von strukturierten Datenelementen und elementaren Werten in eine mit einer Nachricht übertragbaren externen Datendarstellung. Encodierung eines Bit-Strings
- **Un-Marshalling:** Prozess der Erstellung elementarer Werte aus ihren externen Datendarstellung und den Wiederaufbau der ursprünglichen Datenstrukturen. Dekodierung eines Bit-Strings

Formen

- **Nicht-Selbstbeschreibend:** *Sender und Empfänger haben Kenntnis* über die Reihenfolge und Typen der Datenelemente in einer Nachricht
- **Selbstbeschreibend:** Informationen über Reihenfolge und die Typen der Datenelemente sind *in der Nachricht*

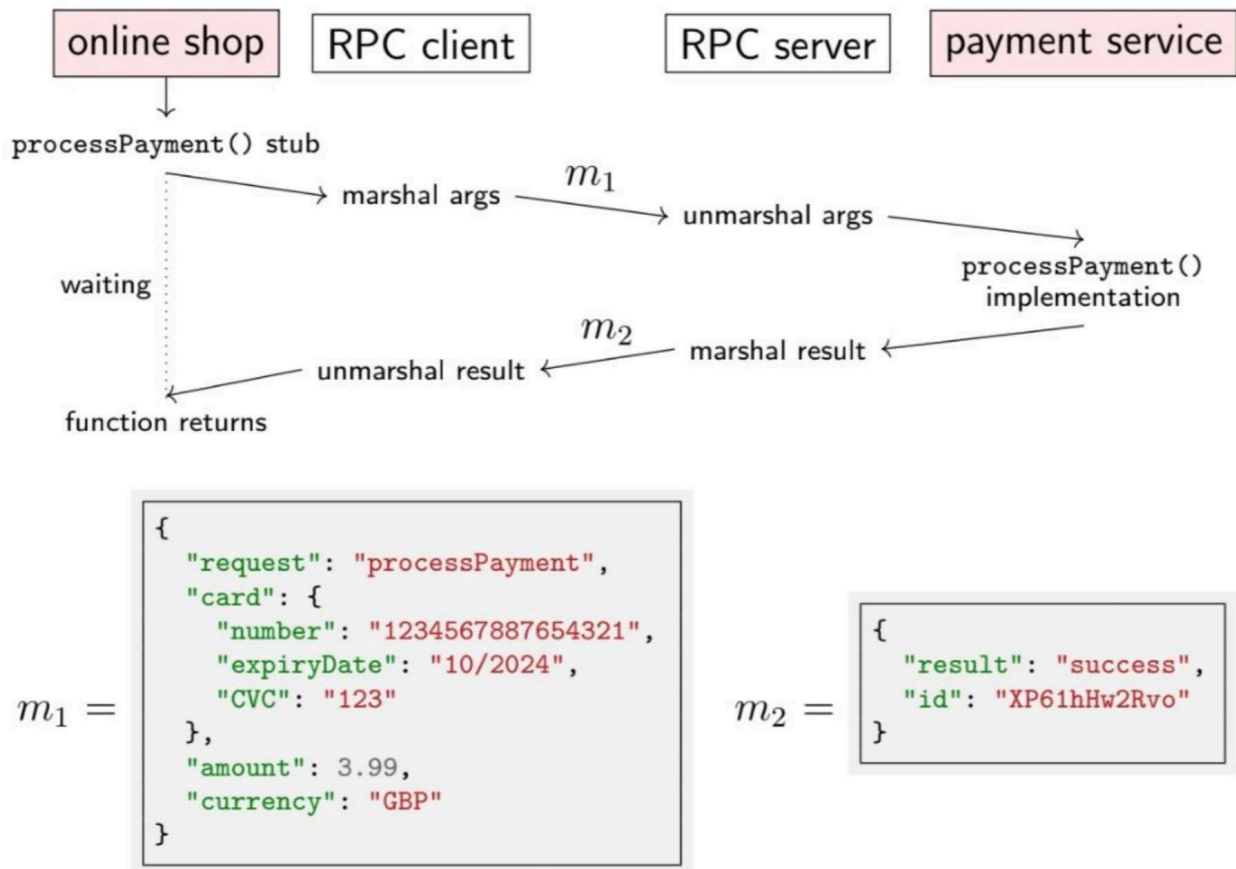
Kommunikationsmuster

Blockierende Synchroner Interaktion

- Typisches Anfrage-Antwort/Request-Reply Protokoll
- Typisch für Verbindungsorientierte Implementierung in Sockets
- Punkt zu Punkt / ohne Message Router
- Ablauf:
 - Client schickt Nachricht an Server und wartet
 - Server empfängt Nachricht und führt die Aktion durch
 - Server sendet Antwort an Client zurück
- Beide müssen zum Zeitpunkt der Interaktion verfügbar sein
Bsp: HTTP

Remote Procedure Call (RPC)

Remote Funktion sieht aus wie **lokaler Funktionsaufruf**.



Ablauf

1. Client ruft Stub auf
2. Parameter werden **marshalled**
3. Nachricht gesendet
4. Server unmarshalled
5. Funktion wird ausgeführt
6. Ergebnis wird zurückgesendet

Typische Fehlerursachen in Verteilten Systemen

- Partition des Netzwerks (Unterbrechungen zwischen Client und Server)
- CPU/Hardware Probleme
- Betriebssystemfehler
- Applikationsfehler
 - Abstürzen
 - Deadlocks
 - nurnoch partielle Funktionalität vorhanden

Kompensierung

- Wiederholung der Anfrage
 - At least once semantik -> Scheitert sobald das Ergebnis nicht idempotent ist

- At most once Semantik -> Server führt Liste über Clientanfragen, löscht Duplikate und Antwortet nur einmal
- Exactly once braucht komplexe Protokolle
 - funktioniert bei "normalem" Request/Reply gar nicht (benötigt Response) -> Blockierend
 - ABER gut für Queues, die auch mal eine Woche warten

Fehlersemantik	Fehlerfreier Ablauf	Nachrichten Verluste	Ausfall des Servers
Maybe	Ausführung: 1 Ergebnis: 1	Ausführung: 0/1 Ergebnis: 0	Ausführung: 0/1 Ergebnis: 0
At-least-once	Ausführung: 1 Ergebnis: 1	Ausführung: ≥ 1 Ergebnis: ≥ 1	Ausführung: ≥ 0 Ergebnis: ≥ 0
At-most-once	Ausführung: 1 Ergebnis: 1	Ausführung: 1 Ergebnis: 1	Ausführung: 0/1 Ergebnis: 1
Exactly-once	Ausführung: 1 Ergebnis: 1	Ausführung: 1 Ergebnis: 1	Ausführung: 1 Ergebnis: 1

Two Generals Problem

Unmöglich alleine durch Nachrichten über einen unzuverlässigen Kanal absolute Gewissenheit über ein gemeinsames Ergebnis zu erreichen
Bzw. Perfekter Konsenz ist unmöglich, wenn Nachrichten verloren gehen

Ein General sendet eine Nachricht: „Angriff um 9 Uhr“.

Der Empfänger bestätigt mit einer Nachricht.

Aber:

- Der erste General weiß **nicht**, ob die Bestätigung angekommen ist.
- Der zweite weiß **nicht**, ob seine Bestätigung bestätigt wurde.
- Dieses Problem setzt sich **unendlich fort**.

Byzantine Generals Problem

Erweiterung von Two Generals: Einer der Generäle könnte ein Verräter sein und falsche Nachrichten übermitteln bei Propagation von Befehlen

Analog: Fehlerhafter oder gehackter Knoten

Es braucht $3f + 1$ ehrenwerte Generäle/Knoten um f bösartige zu tolerieren

Ablauf

- Jeder General ist entweder ehrenwert oder bösartig
- Bis zu f Generäle können boshaft sein
- Ehrenwerte Generäle wissen nicht, wer bösartig ist
- Die bösartigen Generäle können miteinander Kollaborieren

- Trotz dieser Situation müssen die ehrenwerten Generäle zu einer Entscheidung kommen

Theorem

Es braucht $3f + 1$ ehrenwerte Generäle/Knoten um f böartige zu tolerieren

Eigenschaften eines geeigneten Systemmodells

→ Grundlage für das Design Verteilter Algorithmen

Netzwerkverhalten

- **Reliable**
 - Zuverlässige Verbindung: Nachrichten intakt, aber nicht zwangsläufig in der richtigen Reihenfolge
- **Fair-Loss-Link**
 - Nachrichten können verlorgen gehen, dupliziert werden, oder in geänderter Reihenfolge auftauchen
- **Arbitrary** (kompromittierbare) Verbindung
 - Böshafter Agent kann Nachrichten beliebig beeinflussen
- **Netzwerkpartitionierung**
 - Unterbrechungen zwischen der Verbindung zwischen Knoten für unbestimmte Zeit

Verhalten der Knoten

- Crash-Stop
 - Programm stürzt ab und bleibt down
- Crash-Stop-with Recovery
 - Programm stürzt ab und kann wiederhergestellt und neu gestartet werden
- Crash-with-Amnesia
 - Programm stürzt ab und kann neu gestartet werden, verliert aber den Zustand
- Fail-Stop
 - Die ganze Maschine ist down und das Versagen wird kommuniziert
- Omission Errors:
 - Applikationselemente erhalten die Nachrichten führen aber nicht die Assoziierten Aktionen aus, obwohl sie ansonsten Lebendig sind
- Byzantine Error
 - Maschinen oder Teile von Maschinen, Netzwerken oder Anwendungen fallen in unvorhersehbarer Weise aus und führen evt. nur partielle Recovery durch

Zeitliche Annahmen der Kommunikation

- Synchron

- Latenz ist immer niedriger als eine bekannte Grenze
- Knoten bearbeiten Nachrichten mit bekannter Geschwindigkeit
- Partial Synchron
 - Asynchron nur für einige endliche (aber unbekannte Anzahl) von Zeitperioden, sonst synchron
- Asynchron:
 - Nachrichten können beliebig verzögert sein
 - Verarbeitung kann beliebig lang dauern

Maßnahmen für Fehlerumgang - Nutzerseite

Nutzerseite

- Timeout & Retry
- clientseitiges Informationscaching
 - gut für Fehlertoleranz
 - Offlinefähigkeit

Maßnahmen für Fehlerumgang - Serverseite

Fehlerdetektor

- Perfekter Fehler Detektor: Markiert fehlerhafte Systeme, sobald sie fehlerhaft sind (z.B. Crashstop)
 - Problem: schwer zu erkennen, was der Fehler ist (Knoten, Netzwerk etc)
- Beide funktionieren nur in synchronen Knoten mit guter Kommunikationsstruktur

Ping-Acknowledge Protokoll (Ping)

Regelmäßig Ping Pakete von Knoten A zu Knoten B, wenn B antwortet, lebt er noch

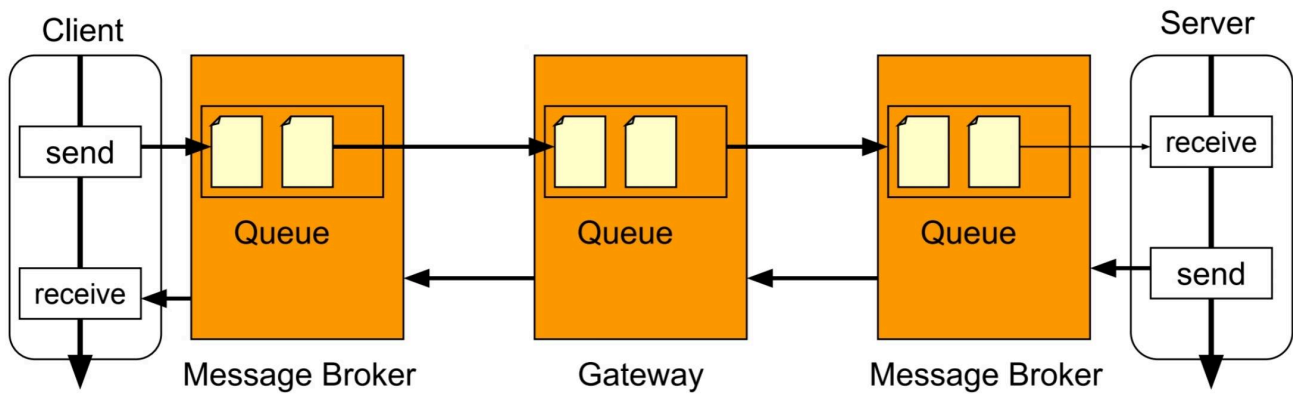
Heartbeat Protokoll

Knoten B meldet sich alle X Zeiteinheiten mit einem Ping/Heartbeat
Wenn A seit X+1 keinen erhalten hat, geht er davon aus, dass B tot ist

In partiell synchronen Systemen

- Heartbeat und Ping erreichen nur "fast" perfekt
 - false positives
 - false negatives
 - meistens funktioniert
- Fehleinschätzungen lassen sich durch andere Mechanismen, wie Gateway & Circuit Breaker ausgleichen

Message Oriented Middleware (MOM)



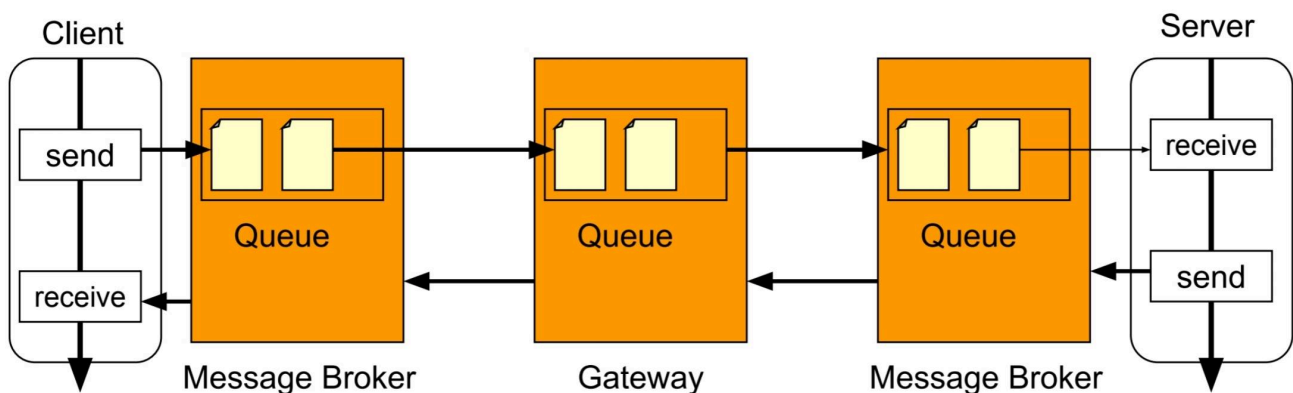
Ablauf

- Client sendet Nachricht
- Client wartet NICHT auf Antwort, sondern arbeitet währenddessen weiter

Nicht blockierender Aufruf

- Client wartet, bis die Anfrage beim Server vollständig angekommen ist
- Arbeitet danach weiter

MOM



Client → Queue → Broker → Server

Vorteile:

- lose Kopplung
- bessere Fehlertoleranz, durch entkopplung vom Server und Client
- Client und Server müssen nicht gleichzeitig online sein
- Förderung von Modularem Design

Broker:

- Komponente zum Transport und Zwischenspeicherung von Nachrichten
- Nutzt Nachrichtenwarteschlangen
- Leitet Nachrichten an Empfänger weiter
- Empfänger müssen im Broker registriert sein

Messaging Modelle

Queue (Point-to-Point)

- mehrere Sender
- **ein Empfänger**

Publish-Subscribe - Topic

- Publisher sendet Nachricht
- alle Subscriber erhalten sie

Broadcast Kommunikation

Broadcast = Nachricht an **alle Knoten einer Gruppe**.

Eigenschaften:

- Peer-to-Peer
- skalierbar
- fehlertolerant